A Dynamically Adaptive Approach to Reducing Strategic Interference for Multiagent Systems

Wei Pan, Nanding Wang, Chenxi Xu, and Kao-Shing Hwang, *Senior Member, IEEE*

***Abstract*—Multiagent reinforcement learning (RL) is widely used and can successfully solve many problems in the real world. In the multiagent RL system, a global critic network is used to guide each agent's strategy to update to learn the most beneficial strategy for the collective. However, the global critic network also makes the current agent's learning be affected by other agents' strategies, which leads to unstable learning. To solve this problem, we propose dynamic decomposed multiagent deep deterministic policy gradient (DD-MADDPG): a new network that considers both global and local evaluations and adaptively adjusts the agent's attention to the two evaluations. Besides, the use of the experience replay buffer by multiagent deep deterministic policy gradient (MADDPG) produces outdated experience, and the outdated strategies of other agents further affect the learning of the current agent. To reduce the influence of other agents' outdated experience, we propose TD-Error and Time-based experience sampling (T2-PER) based on DD-MADDPG. We evaluate the proposed algorithm's performance according to the learning stability and the average return obtained by the agents. We have conducted experiments in the MPE environment. The results show that the proposed method has better stability and higher learning efficiency than MADDPG and has a certain generalization ability.**

***Index Terms*—Multiagent, reinforcement learning (RL), strategic interference.**

I. Introduction

REINFORCEMENT learning (RL) can learn the optimal strategy by interacting with the environment to form an autonomous learning system [1], [2]. So far, deep RL can solve many complex problems, such as computer games [3]–[5] and robotic tasks [6], [7]. Unfortunately, when faced with complex tasks in the real world, such as urban traffic control [8], garbage collection [9], network packet routing [10], etc., traditional deep RL does not perform well. To solve the problem of multiagent collaboration, Lowe *et al.* [11] proposed multiagent deep deterministic policy gradient (MADDPG) based on deep deterministic policy gradient (DDPG). This method usually uses a centralized training-distributed execution method for learning, which greatly reduces the learning cost and improves the agent's learning efficiency. The main idea is that all agents learn with a shared centralized value function (CVF) during training, and each agent acts in a decentralized way during the execution phase. As the environmental agent of each agent, CVF can easily understand each agent's value by adding an appropriate credit distribution mechanism.

Although the mode of centralized training decentralized execution has been popular in multiagent learning recently [12]–[14], due to the existence of CVF in the centralized training framework, the current learning is always affected by other agents' strategies in the process of multiagent training [11]: assuming that the current agent's strategy is excellent, it should get a higher evaluation. However, if the other agents' strategy is bad, the overall evaluation will be lower because of the global critic network used by the centralized training framework. It is unreasonable to use this evaluation to guide the current agent to update the strategy. This can affect agent learning stability, and sometimes it is difficult to find the optimal strategy, which is a serious problem in the current multiagent deep RL (MADRL). Decomposing the CVF into a single value function can reduce the influence of the other agents on the current agent. For example, VDN [15] proposed a value decomposition method, which decomposes the central value function into single-agent value functions, and the agents choose actions according to the local agent value function. It effectively reduces the strategy interference of other agents. QMIX [16] is proposed on the basis of VDN, which transforms the mapping relationship between the single-agent value function to the central value function from the original linear mapping to the nonlinear mapping, which improves the performance of the model. However, both VDN and QMIX assume that the optimal joint action is equivalent to the set of optimal actions of each agent, but this assumption does not conform to all models, so QTRAN [17] continued on the basis of VDN and QMIX. It proposed a new value-function decomposition method does not introduce other assumptions and restrictions, and is suitable for a wider range of problems. However, the above value-function decomposition methods are usually used in environments where only global returns can be obtained but individual returns cannot be obtained. However, most environments are artificially designed, and individual returns are easy to obtain in reality. We can directly train a single-agent Q network using the individual returns. There is no need to

Manuscript received 12 April 2021; revised 22 July 2021; accepted 5 September 2021. Date of publication 22 September 2021; date of current version 9 December 2022. This work was supported in part by the National Defense Science and Technology Key Laboratory Fund under Grant 6142103190308, and in part by the Fundamental Research Funds for the Central Universities under Grant G2019KY05201. *(Corresponding author: Kao-Shing Hwang.)*

Wei Pan, Nanding Wang, and Chenxi Xu are with the School of Computer Science, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: panwei@nwpu.edu.cn; 2798197856@qq.com; 2942486815@qq.com).

Kao-Shing Hwang is with the Department of Electrical Engineering, National Sun Yat-sen University, Kaohsiung 80424, Taiwan, and also with the Department of Healthcare Administration and Medical Informatic, Kaohsiung Medical University, Kaohsiung 80708, Taiwan (e-mail: hwang@g-mail.nsysu.edu.tw).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCDS.2021.3110959.

Digital Object Identifier 10.1109/TCDS.2021.3110959

2379-8920 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

obtain a single value function by means of value-function decomposition. In an environment, where individual returns can be obtained, using the decentralized training-decentralized execution [18] framework to train each agent independently can reduce the impact of other agents' strategies on the current agent, and fundamentally solve the problem of agents' strategy interference. However, this assumption will make the agent pay more attention to the impact of the actions taken on itself and ignore the impact on the collective, which is not conducive to agent cooperation and will cause the agent difficult to converge to the optimal strategy. Sheikh and Bölöni [19] used two critical networks to separately evaluate the impact of the agents' current actions on individual and collective returns, achieving a balance between individual and collective returns. Inspired by [19], we consider whether we can use two critical networks to evaluate the current actions collectively and individually, and jointly guide the agents' strategies to update, considering the influence of the agents' actions on the individual and the collective, while ensuring the cooperation of agents and reducing the interference of agents' strategies? In order to optimize our method, we consider to realize the adaptive transformation of the strategy networks' attention of global critic and local critic. We have noticed that generative adversarial selfish networks (GASNet) realizes the automatic switch of selfishness and selflessness: using graph neural networks (GNNs) to construct an adjacency matrix to describe whether the agent is selfish or selfless. The influence coefficients of the selfishness matrix and the selflessness matrix on the social graph are α and $1-\alpha$, respectively, where α is the selfish coefficient. The social graph switches between selfishness and selflessness by adjusting α. We consider applying GASNet's idea of realizing the automatic switch of selfishness and selflessness to our method to realize the adaptive adjustment of global critic and local critic.

Although the above method reduces the influence of the other agents' strategies on the current agent's learning, it has not considered the influence of the other agents' outdated experience. MADDPG uses the experience replay buffer to store experience and extracts experience from the experience replay buffer to train the Q network, improving training efficiency and learning stability. But in a multiagent system, the experience of the agents is constantly changing. When other agents' strategies change, the current agent should adjust its strategy according to its strategies. If the outdated experience is used, it will lead to the agents' unstable training and difficulty in converging to the optimal strategy. To reduce the influence of the outdated experience, some MADRL methods do not use the buffer [20] or use the small-capacity replay buffer [21] to discard outdated experience. However, this reduces the utilization rate of experience samples, increases the interaction between the agent and the environment, extends the learning time, and makes it difficult for the agent to learn the optimal strategy. Besides, this also reduces the diversity of experience samples so that the agent can only learn the experience in short episodes, resulting in unstable training at last. The hysteretic Q-learning [22] and leniency [23] were proposed to deal with nonstationarity due to concurrent learning of multiple agents in multiagent systems. These methods can reduce the influence of other agents' outdated experiences and are more likely to converge to the optimal policies. The hysteretic Q-learning and leniency were successfully applied to MADRL (HDQN) [24], (LDQN) [25], and it was proved that in a fully cooperative environment with random rewards, LDQN is more likely to converge to the optimal strategy than HDQN. However, LDQN performs a hash mapping on the state in the process of determining the lenient value, and maps the continuous state to the discrete value, which brings a certain deviation. Moreover, when the state action space is continuous, the hash function is difficult to determine, which brings greater difficulties to the application of leniency to MADDPG. Wang and Zhang [26] proposed a prioritized experience replay method to solve the problem of incompatibility between outdated strategies and the current agent's strategy in a continuous action environment and proved the effectiveness of the method through covert communication experiments. The experimental results on the covert communication problem confirm the effectiveness of our proposed method. This method uses TD error to determine the sample priority and determines the sample's probability of being sampled based on the priority. When the previous experience samples deteriorate the agent from the optimal decision, the TD error of the samples recently added to the experience buffer will increase; on the other hand, when the neural network is overfitting, the probability of extracting the previous experience samples will increase. However, this method is still unreasonable enough to deal with outdated experiences because they may have a large TD error.

This article has made further optimization based on previous research to reduce the other agents' strategies interference. We proposed dynamic decomposed MADDPG (DD-MADDPG) based on MADDPG: a new multiagent learning framework that uses global evaluation and local evaluation to guide agents to update strategy and can adaptively realize the combination of individual learning and team cooperation, reduces interference of other agents, improves the stability of agent training, and optimizes agents' strategies. To reduce the impact of the experience replay buffer, borrowing the idea of Wang and Zhang [26], we propose a priority experience replay (T2-PER) based on TD error and sample time based on DD-MADDPG. This method determines the sample priority based on the TD error and the sample time. It determines the probability of the sample being sampled based on the priority, which reduces the probability of outdated experience being selected, thereby effectively reducing the impact of other agents' outdated experience.

Our contributions are as follows.

We proposed a new framework for multiagent learning, named DD-MADDPG. This framework uses two critic networks to maximize the optimization of agent strategy, considers the global evaluation and local evaluation of the agent's strategy simultaneously, and dynamically updates the agent's strategy so that the agent can cooperate with other agents while reducing the impact of other agents' strategies on the current agent training.

Based on DD-MADDPG, a prioritized experience replay (T2-PER) based on TD error and sample time is proposed. In DD-MADDPG, to update the global critic network, global

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

rewards are calculated. Here, the TD error of the empirical sample is calculated based on the global rewards. Since MADDPG does not calculate the global rewards, this is not feasible in the MADDPG algorithm.

We evaluated DD-MADDPG and T2-PER in the MPE [11] simulation environment. During the experiment, we used the Adam optimizer to train the neural network. The method is simple and efficient and can adaptively adjust the learning rate during the training process to achieve smooth convergence [27]. The results show that, compared with MADDPG, DD-MADDPG has better performance: It is more stable and can obtain higher rewards, T2-PER can further improve the learning stability based on DD-MADDPG. We also expanded the number of agents in the simulation environment. The results show that DD-MADDPG and T2-PER still have better performance than MADDPG when there are more agents. This shows that the proposed method has a certain generalization ability.

II. Background

A. Deep Deterministic Policy Gradient

DDPG is an actor–critic algorithm [18], which uses the deep neural network to approximate the deterministic policy $\pi(o) : O \rightarrow A$ parameterized by θ and the critic function $Q^{\pi} : O, A \rightarrow$ value parameterized by φ. DDPG is an off-policy algorithm using experience replay and a target network to improve data utilization and stabilize training. The algorithm maximizes the objective function

$$\nabla J(\theta) = E_{o,a\sim D}\left[\nabla_{\theta}\pi(a|o)\nabla_{a}Q^{\pi}(o,a)\right] \tag{1}$$

by optimizing the parameter of the network, where D denotes the replay buffer.

B. Multiagent Deep Deterministic Policy Gradient

The MADDPG is a variant of DDPG. Different from DDPG, in this algorithm, the input of the critic network is not state–action (o, a), but the joint observation-action $(o_1, o_2, \ldots, o_N, a_1, a_2, \ldots, a_N)$. Agent i will be updated as follows:

$$y_i\left(r_i, \boldsymbol{o'}\right) = r_i + \gamma Q'_i\left(\boldsymbol{o'}, \boldsymbol{a'}\right) \tag{2}$$

$$\delta_i\left(\boldsymbol{o}, \boldsymbol{a}, r_i, \boldsymbol{o'}\right) = y_i\left(r_i, \boldsymbol{o'}\right) - Q_i(\boldsymbol{o}, \boldsymbol{a}) \tag{3}$$

$$\nabla_{\theta_i^Q} L\left(\theta_i^Q\right) = E_{\boldsymbol{o},\boldsymbol{a},r_i,\boldsymbol{o'}\sim D}\left[\nabla_{\theta_i^Q}\delta_i^2\left(\boldsymbol{o}, \boldsymbol{a}, r_i, \boldsymbol{o'}\right)\right] \tag{4}$$

$$\nabla_{\theta_i^{\pi}} L\left(\theta_i^{\pi}\right) = E_{\boldsymbol{o},\boldsymbol{a}\sim D}\left[\nabla_{\theta_i^{\pi}}\pi_i(o_i)\nabla_{a_i}Q_i(\boldsymbol{o}, \boldsymbol{a})\right] \tag{5}$$

where $\boldsymbol{o} = (o_1, \ldots, o_i, \ldots, o_N)$ and $\boldsymbol{a} = (a_1, \ldots, a_i, \ldots, a_N)$ are the joint observation and the joint action, respectively. $\boldsymbol{o'} = (o'_1, \ldots, o'_i, \ldots, o'_N)$ and $\boldsymbol{a'} = (a'_1, \ldots, a'_i, \ldots, a'_N)$ are the joint observation and the joint action at the next step, where $a'_i = \pi'_i(o'_i)$; and o_i and a_i denote the private observation and action of agent i, respectively, and Q'_i and π'_i denote the target critic network and the target policy network of agent i, respectively.

III. Algorithmic Contributions

A. Overall Algorithm Framework

We propose a DD-MADDPG to reduce the impact of other agents' strategies on the current agent's learning. But DD-MADDPG does not consider the impact of time changes, so a priority experience replay (T2-PER) based on TD error and time is proposed to reduce the influence of other agents' outdated experience on the current agent's training based on DD-MADDPG. In the following, we describe our main algorithmic contributions.

Fig. 1 is an overview of the overall structure of DD-MADDPG and T2-PER. Our model is based on a multiagent training model of centralized training and decentralized execution. The agent system obtains the joint observation $\boldsymbol{o}$ from the environment at time t, and each agent $\boldsymbol{i}$ obtains the individual observation o_i from the joint observation, and obtains action a_i according to the individual strategy π_i. The multiagent system executes the joint action $\boldsymbol{a}$ to enter the next state $\boldsymbol{o'}$ and obtains the global return $\boldsymbol{r_g}$ and the local return $\boldsymbol{r_l}$, where $\boldsymbol{o} = (o_1, \ldots, o_i, \ldots, o_N)$, $\boldsymbol{a} = (a_1, \ldots, a_i, \ldots, a_N)$, $\boldsymbol{o'} = (o'_1, \ldots, o'_i, \ldots, o'_N)$, and $\boldsymbol{r_l} = (r_1, \ldots, r_i, \ldots, r_N)$. N represents the number of agents. The agent interacts with the environment to generate experience: $(\boldsymbol{o}, \boldsymbol{a}, \boldsymbol{r_l}, \boldsymbol{r_g}, \boldsymbol{o'})$, we calculate the priority for each experience, and then get a sample with priority: $(\boldsymbol{o}, \boldsymbol{a}, \boldsymbol{r_l}, \boldsymbol{r_g}, \boldsymbol{o'}, \boldsymbol{p})$. We put the generated samples into the experience replay buffer and select samples based on sample priority. The sampled experience will be used to train the global critic network and the local critic network. The global critic network and the local critic network jointly guide the update of the strategy network. The strategy network's attention to the evaluation of the two critic networks changes dynamically with the training process. The agent can continue to learn through interaction with the environment to complete the task through the above process.

B. Dynamic Decomposed Multiagent Deep Deterministic Policy Gradient

We propose DD-MADDPG: a self-adaptive MADRL algorithm that uses two critic networks to evaluate the policy network together, which enables the agent to cooperate with other agents while reducing the influence of other agents' strategies on the current agent training. The policy network's attention to the two critic networks changes dynamically with the training process, making the agent complete the transition from individual learning to collective learning adaptively and optimize the agent strategy to the maximum.

Unlike MADDPG, which uses a critic network to guide the agent's strategy update, we use a dual network to guide the agent's strategy to update together. We have added a local critic network based on MADDPG to evaluate the influence of the agent's current strategy on itself. The original critic network in MADDPG is named the global critic network here and is used to evaluate the agent's current strategy's impact on the collective. We use two networks to jointly evaluate the strategy network and adaptively realize the organic combination of individual learning and cooperation with other agents. At the beginning of the training, due to the agent's lack of experience,

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

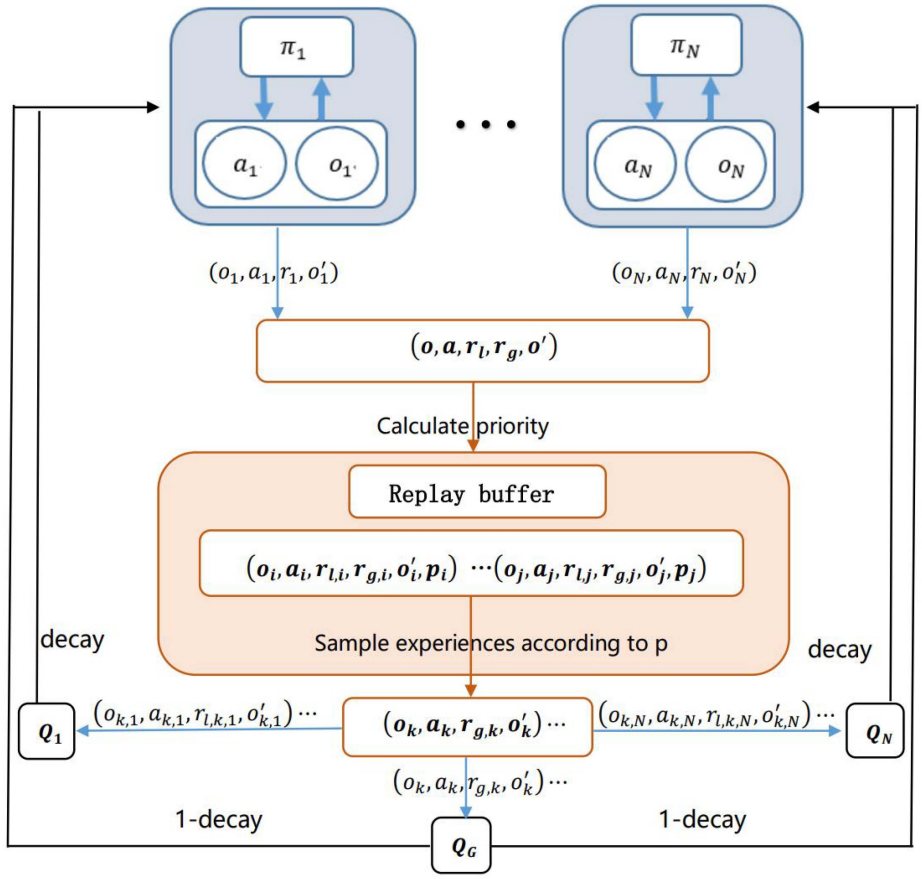


Fig. 1. Overview of the DD-MADDPG and T2-PER architecture.

the agent pays more attention to the local critic network while paying less attention to the global criticism network. As the training process progresses, the agent will have more experience and can better complete the individual task, then the agent gradually reduces its attention to the local critic network and continues to increase its attention to the global critic network.

Concretely, we consider an environment with N agents with policies $\pi = \{\pi_1, \ldots, \pi_N\}$ parameterized by $\theta = \{\theta_1, \ldots, \theta_N\}$. Then, the gradient of the expected return for i can be written as

$$\nabla J(\theta_i) = E\left[\nabla_{\theta_i}\pi_i(a_i|o_i)\nabla_{a_i}Q_i^{\pi}(\boldsymbol{o}, \boldsymbol{a})\big|_{a_i=\pi_i(o_i)}\right] \tag{6}$$

where $Q_i^{\pi}(\boldsymbol{o}, \boldsymbol{a})$ is a centralized action-value function that takes the actions of all the agents $\boldsymbol{a}$, where $\boldsymbol{a} = (a_1, \ldots, a_i, \ldots, a_N)$ and the joint observation $\boldsymbol{o}$, where $\boldsymbol{o} = (o_1, \ldots, o_i, \ldots, o_N)$, and output the Q-value for agent i.

Now, we can extend MADDPG to DD-MADDPG. We can write the modified policy gradient for each agent i as

$$\begin{aligned}\nabla J(\theta_i) = {} & w_1 * E_{\boldsymbol{o},\boldsymbol{a}\sim D}[\nabla_{\theta_i}\pi_i(a_i|o_i)\nabla_{a_i}Q_{\varphi}^{g}(\boldsymbol{o}, \boldsymbol{a})] \\ & + w_2 * E_{o_i,a_i\sim D}\left[\nabla_{\theta_i}\pi_i(a_i|o_i)\nabla_{a_i}Q_i^{\pi}(o_i, a_i)\right]\end{aligned} \tag{7}$$

where $a_i = \pi_i(o_i)$ is action from agent i following policy π_i and D is the experience replay buffer. w_1 and w_2 are the weights of the global critic network and the local critic network, respectively, representing the degree of attention the policy network pays to the two critic networks. The initial value of w_2 is 1 and it decays with the training process

$$w_2 = w_2 * \text{decay} \tag{8}$$

$$w_1 = 1 - w_2 \tag{9}$$

where decay is a hyperparameter, decay < 1.The centralized action-value function Q_{φ}^{g} is updated as

$$L(\varphi) = E_{\boldsymbol{o},\boldsymbol{a},r,\boldsymbol{o'}}\left[\left(Q_{\varphi}^{i}(\boldsymbol{o}, \boldsymbol{a}) - y_g\right)^2\right] \tag{10}$$

where y_g is defined as

$$y_g = r_g + \gamma Q_{\varphi'}^{g}(\boldsymbol{o'}, \boldsymbol{a'}) \tag{11}$$

where $\boldsymbol{o'} = (o'_1, \ldots, o'_i, \ldots, o'_N)$, $\boldsymbol{a'} = (a'_1, \ldots, a'_i, \ldots, a'_N)$, and $a'_i = \pi'_i(o'_i)$. $\boldsymbol{\pi'} = (\pi'_1, \ldots, \pi'_N)$ is a set of target policies parameterized by $\theta' = (\theta'_1, \ldots, \theta'_N)$. The local action-value function Q_i^{π} is updated as

$$L(\emptyset_i) = E_{\boldsymbol{o},\boldsymbol{a},r,\boldsymbol{o'}}\left[(Q_i^{\pi}(o_i, a_i) - y_1)^2\right] \tag{12}$$

where y_1 is defined as

$$y_1 = r_1^i + \gamma Q_{\emptyset'_i}^{\pi'}(o'_i, a'_i)|_{a'_i=\pi'_i(o'_i)}. \tag{13}$$

Although the DD-MADDPG method can reduce the influence of other agents' strategies on the current agent learning to a certain extent, it does not consider the influence of agents' strategies over time. In a multiagent system, the agent's

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

strategies are constantly changing. Using previous outdated experiences to train the agent will make it difficult for the agent to converge to the best strategy. To reduce the probability that the agent chooses outdated experience, we propose an experience sampling method based on TD-Error and sample time to replace the traditional random sampling.

C. TD-Error and Time-Based Experience Sampling

We propose a prioritized experience replay based on TD error and sample time. It determines the sample priority based on the TD error of the experience and the time the sample is placed in the experience replay buffer. It then determines the probability of the experience being selected based on the sample priority. This can improve the training speed while reducing the influence of outdated experience on the agent training, thereby optimizing the agent strategy to the greatest extent and improving training stability.

Concretely, we define the probability of the experience i be selected as

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \tag{14}$$

where $p_i > 0$ is the priority of experience sample i. The exponent α denotes how much prioritization is used, with $\alpha = 0$ corresponding to the uniform case.

So, how do we determine the sample priority? The prioritized experience replay method based on TD error is widely used because it can filter out samples to make the strategy converge faster. Still, it is often used in single-agent systems. Due to the continuous changes of agents' strategies in multiagent systems, the staleness of experience samples is an important issue. Whether the samples are out of date does not correlate with the TD error. The outdated samples may have a larger TD error. It is unreasonable to assign higher priority to these samples.

So, we consider the influence of sample time when determining sample priority. Concretely, the sample time is determined by the time when the experience sample is put into the experience replay buffer. The later the sample is put, the larger the sample time, thereby increasing the priority of later experience and reducing the priority of earlier experience. The TD error is calculated based on the global rewards and global Q-Value mentioned in the DD-MADDPG algorithm.

We define the priority of the experience as

$$p_i = \delta_i^2 * (1 - e^{-K*T_i}) \tag{15}$$

where T_i is time of experience sample i. The initial value of T is 1 and it keeps increasing with the training process. K is an impact factor, $K = 0.95$, and it indicates the degree of influence of time on the priority of the experience.

The TD error δ_i of the ith experience is defined as

$$\delta_i = r_{g,t+1}^i + \gamma Q_g^i(o_{t+1}, a_{t+1}) - Q_g^i(o_t, a_t) \tag{16}$$

where $r_{g,t+1}^i$ denotes the global reward of experience sample i at time $t+1$, which is defined in DD-MADDPG, γ is the discount factor, and $Q_g^i(o_{t+1}, a_{t+1})$ and $Q_g^i(o_t, a_t)$ denote the global Q-Value of experience sample i at time $t+1$ and t, respectively.

Algorithm 1 DD-MADDPG and T2-PER

1. Definition:
s^t := the joint observation for all agents at time t
o_i^t := the local observation for agent i at time t
a^t := the joint action for all agents at time t
a_i^t := the local action for agent i at time t
π_{θ_i} := the policy for agent i
Q_g := the global action-value function
Q_l := the local action-value function
T := the total training steps
2. Repeat:
for episode=1 to T **do**
  for t=1 to episode-length **do**
    Get Environment state s^t.
    For each agent i,select action $a_i^t = \pi_{\theta_i}(o_i^t)$
    Execute actions $a^t = [a_1^t, \ldots, a_N^t]$
    Receive global reward r_g^t and local rewards r_1^t.
    /* Calculate priority p^t of experience t */
    Calculate TD error according to r_g and Q_g:
    $\delta_t = r_{g,t+1}^t + \gamma Q_g^t(s_{t+1}, a_{t+1}) - Q_g^i(s_t, a_t)$
    Calculate priority according to TD error and time:
    $p_t = \delta_t^2 * (1 - e^{-K*T_t})$
    Store $(s^t, a^t, r_1^t, r_g^t, s^{t+1}, p^t)$ in replay buffer.
  end for
  if episode mod d **then**
    Calculate the probability of the experience i be selected:
    $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$
    Sample minibatch of size S $(s^j, a^j, r_1^j, r_g^j, s'^j, p^j)$ from buffer according to probability.
    $(a'_1, \ldots, a'_N) := \left(\pi'_{\theta_i}\left(o_i'^j\right), \ldots, \pi'_{\theta_N}\left(o_N'^j\right)\right)$
    for agent i=1 to N **do**
      Set $y_1^i = r_1^{i,j} + \gamma Q_{\emptyset'_i}^{\pi'}(o'_{i,j}, a'_{i,j})$
      Update local critic by minimizing
      $\frac{1}{s}\sum_j \left(Q_{\varphi_i}^{\pi}(o_{i,j}, a_{i,j}) - y_1^j\right)^2$
      Set $y_g^j = r_g^j + \gamma Q_\varphi^{g_j}\left(\boldsymbol{o'^j}, \boldsymbol{a'^j}\right)$
      Update global critic by minimizing
      $\frac{1}{s}\sum_j \left(Q_\varphi^{\pi}(\boldsymbol{o}, \boldsymbol{a}) - y_g^j\right)^2$
      Process of updating parameters with Adam:
      while $\theta_{i,t}$ not converged **do**
        Calculate gradient
        $g_{i,t} = \frac{1}{s}\sum_j \nabla_{\theta_i}\pi_i(a_i|o_{i,j})\nabla_{a_i} Q_\varphi^g\left(\boldsymbol{o^j}, \boldsymbol{a^j}\right)$
        $+\nabla_{\theta_i}\pi_i(a_i|o_{i,j})\nabla_{a_i} Q_{\varphi_i}^{\pi}(o_j, a_{i,j})$
        Update biased first moment estimate
        $m_{i,t} = \beta_1 * m_{i,t-1} + (1-\beta_1) * g_{i,t}$
        Update biased second raw moment estimate
        $u_{i,t} = \beta_2 * u_{i,t-1} + (1-\beta_2) * (g_{i,t})^2$
        Compute bias-corrected first moment estimate
        $m'_{i,t} = \frac{m_{i,t}}{1-(\beta_1)^t}$
        Compute bias-corrected second raw moment estimate
        $u'_{i,t} = \frac{u_{i,t}}{1-(\beta_2)^t}$
        Update parameters
        $\theta_{i,t} = \theta_{i,t-1} - \frac{m'_{i,t}}{\sqrt{u'_{i,t}}+\varepsilon} lr$
      end while
    end for
    Update target network parameters
    $\emptyset'_i \leftarrow \tau\emptyset_i + (1-\tau)\emptyset'_i$
    $\varphi'_i \leftarrow \tau\varphi_i + (1-\tau)\varphi'_i$
    $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i$
  end if
end for

The overall algorithm we refer to as DD-MADDPG and TD-Error and Time-based prioritized experience replay (T2-PER) is described in Algorithm 1.

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

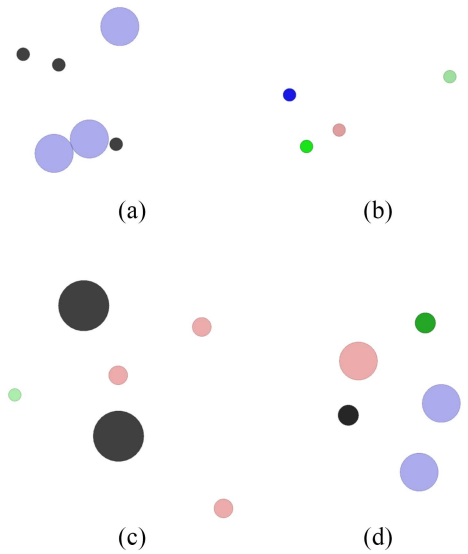


Fig. 2. Overview of the MPE environments used in the experiments. (a) Cooperative navigation. (b) Keep-away. (c) Predator–prey. (d) Physical deception.

IV. Experiments

In this section, we describe our experiment. We apply the proposed DD-MADDPG and T2-PRE to different MPE environments and evaluate the algorithm based on the performance of different algorithms

A. Experimental Environment

The following is an introduction to the experimental environment of this article. We have evaluated the proposed algorithm in the following four MPE environments, as shown in Fig. 2.

1) Cooperative Navigation: There are N agents and N landmarks. Agents are rewarded based on how far any agent is from each landmark. Agents are penalized if they collide with other agents. So, agents have to learn to cover all the landmarks while avoiding collisions.

2) Keep-Away: There are N agents, M adversaries, and one landmark. The agent is rewarded based on the distance to the landmark. The adversary is rewarded if it is close to the landmark and if the agent is far from the landmark. So, the adversary learns to push the agent away from the landmark.

3) Predator–Prey: Good agents (green) are faster and want to avoid being hit by adversaries (red). Adversaries are slower and want to hit good agents. Obstacles (large black circles) block the way.

4) Physical Deception: There are one adversary (red), N good agents (green), and N landmarks. All agents observe position of landmarks and other agents. One landmark is the "target landmark" (colored green). Good agents are rewarded based on how close one of them is to the target landmark, but negatively is rewarded if the adversary is close to the target landmark. The adversary is rewarded based on how close it is to the target, but it does not know which landmark is the target landmark. So, good agents have to learn to "split up" and cover all landmarks to deceive the adversary.

B. Experimental Details

Table I shows the specific parameter settings and network structure in the experiment.

For the neural network structure, the actor network is composed of three layers. The critic network is composed of four layers. Each hidden layer contains 64 units, and the output layer is activated by the ReLU activation function. Both actor network and critic network use the Adam optimizer for training.

C. Evaluations

We mainly evaluate the performance of the proposed algorithm from two aspects: 1) learning stability and 2) global rewards. On the one hand, the overall performance can be directly reflected by the agents' learning curve graph. On the other hand, it can also be measured by the average rewards and confidence interval of the last 10,000 episodes. To verify the generalization ability of the proposed method, we, respectively, performed DD-MADDPG and T2-PER in an environment with a small number of agents (2–3) and an environment with an increase in the number of agents (10 and 50) to evaluate the performance.

In order to evaluate the performance of the DD-MADDPG algorithm, we use the MADDPG and DDPG algorithms as the baseline, by comparing DD-MADDPG with MADDPG, DDPG, TD3 [28] (here, it is applied to the multiagent environment in a centralized manner [29]), and decomposed multiagent deep deterministic policy gradient (DE-MADDPG, without considering the dynamic changes of the attention strategy network paid to the global evaluation and local evaluation.) We use the Adam optimizer to train the neural network in all models, automatically adjusting the learning rate during the gradient descent process to improve training stability.

For the Cooperative navigation environment, we trained for 100 000 episodes; for the keep-away environment, we trained for 50 000 episodes; and for the predator–prey environment and physical deception environment, we trained for 300 000 episodes. We ran ten random seeds in each environment.

Fig. 3 shows the curve of the global rewards of the agent with the training episodes. The light part is the error band of ten random seeds, and the dark part is the average value. Overall, the learning stability and the global reward of the DD-MADDPG algorithm is significantly better than the baseline. For the Keep-away environment, the performance of different methods has no obvious difference—they all quickly converge to almost the same value. A reasonable explanation is that the Keep-away environment is relatively simple, and the training goal can be reached quickly using the baseline algorithms. For the predator–prey environment, the DD-MADDPG algorithm has not yet converged within 300 000 rounds. Still, from the perspective of the agents' global reward, DD-MADDPG is higher than the other methods, and the curve maintains steady growth. It is expected to converge to a higher value. On the whole, DDPG converges faster than MADDPG, but the return is generally lower than MADDPG. We infer this is because the DDPG network is smaller and has fewer parameters than MADDPG, making it easier to converge. However, because

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETERS USED FOR ALL ALGORITHMS IN THE EXPERIMENTS

Parameters	T2-PER	TD-PER	DD-MADDPG	DE-MADDPG	MADDPG	DDPG	TD3	S-BUFFER
Replay buffer	5e5	5e5	5e5	5e5	5e5	5e5	5e5	5e4
Minibatch size	256	256	256	256	256	256	256	256
Lr-a	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4
Lr-gc	1e-3	1e-3	1e-3	1e-3	1e-3		1e-3	1e-3
Lr-lc	1e-4	1e-4	1e-4	1e-4				
Lr-c						1e4		
Decay	0.95	0.95	0.95					
α	1	1						
K	1.01							

TABLE II
GLOBAL REWARD IN THE LAST 10 000 EPISODES OVER TEN SEEDS. ± CORRESPONDS TO 95% CONFIDENCE INTERVAL OVER SEEDS

Environment	DD-MADDPG	DE-MADDPG	TD3	MADDPG	DDPG
Cooperative navigation	-528.11 ± 1.57	-706.20 ± 2.96	-1510.89 ± 5.95	-1536.87 ± 5.64	-3426.59 ± 6.07
Keep-away	81.71 ± 0.18	80.72 ± 0.41	75.93 ± 0.52	79.37 ± 0.65	78.17 ± 0.52
Predator-prey	1003.00 ± 0.34	885.7 ± 0.37	880.00 ± 0.26	688.70 ± 0.28	795.2 ± 0.26
Physical deception	-983.27 ± 5.42	-1487.44 ± 12.07	-9541.72 ± 22.49	-10087.25 ± 26.32	-12907.82 ± 21.24

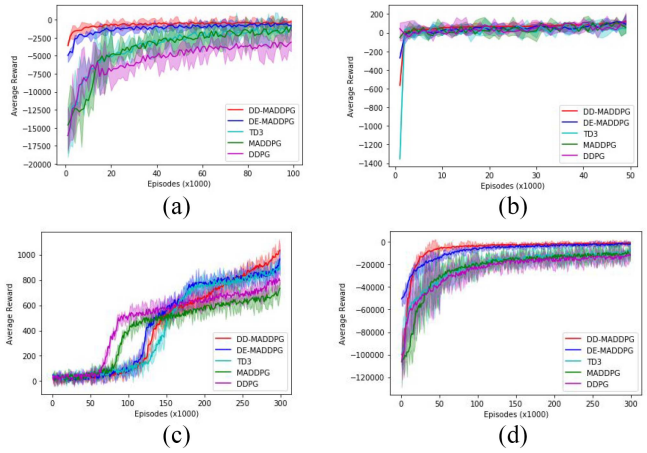


Fig. 3. Graph of average global rewards with training episodes. (a) Cooperative navigation. (b) Keep-away. (c) Predator–prey. (d) Physical deception.

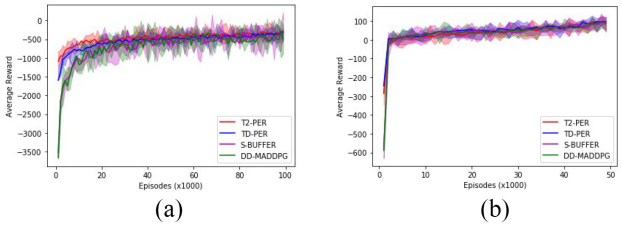


Fig. 4. Graph of average global rewards with training episodes. (a) Cooperative navigation. (b) Keep-away.

DDPG does not consider the cooperation between agents, it is more inclined to converge to suboptimal than MADDPG. DD-MADDPG combines the advantages of DDPG's faster convergence and MADDPG's ability to converge to a higher value to achieve better performance.

Fig. 3 visually reflects the performance of different methods, but it does not quantitatively explain the performance improvement of different methods. Therefore, we calculate the average and confidence interval of each method's last 10 000 episodes in all environments, as shown in Table II. The performance of DD-MADDPG algorithm is significantly better than other methods in the environments except for the Keep-away environment. Especially, in the Cooperative navigation environment, the joint return of the DD-MADDPG algorithm is increased by approximately 67% compared to MADDPG and about 25% compared to DE-MADDPG. The standard error is reduced by 71% compared to MADDPG and 47% compared to DE-MADDPG.

We evaluated the T2-PER algorithm in two environments: 1) cooperative navigation and 2) keep-away (because the Predator–prey and Physical deception environments are relatively complex, it is not conducive to further experiments, and these two environments and Keep-away both belong to Competitive environment, so we no longer use these two environments to evaluate algorithms). Here, we use the DD-MADDPG algorithm as a baseline to evaluate the effectiveness of T2-PER by comparing T2-PER with TD-PER (it does not consider the experience time of T2-PER, only determines the sample priority based on the TD error), S-BUFFER (small buffer [21], it eliminates outdated experience by reducing the capacity of the buffer. We reduce the buffer to 1/10 of the original) and DD-MADDPG. Training episodes are the same as above. Fig. 4 is the learning curve.

According to Fig. 4(a), we found that using T2-PER can make the strategy converge faster. It can make the strategy converge in 10 000 episodes, while DD-MADDPG and S-BUFFER converge after about 20 000 episodes. The S-BUFFER method has no obvious changes compared with DD-MADDPG. A reasonable explanation is that S-BUFFER reduces the impact of outdated samples by eliminating outdated experience, but it also reduces the sample diversity, which is not conducive to the improvement of learning stability. To quantitatively evaluate the performance improvement of T2-PER, we calculated the average reward and confidence

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

TABLE III
GLOBAL REWARD IN THE LAST 10 000 EPISODES OVER TEN SEEDS. ± CORRESPONDS TO 95% CONFIDENCE INTERVAL OVER SEEDS

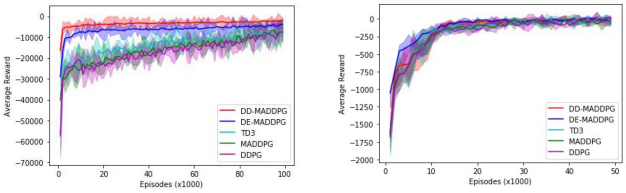
Environment	T2-PER	TD-PER	S-BUFFER	DD-MADDPG
Cooperative navigation	-382.26 ± 1.23	-406.54 ± 1.40	-400.71 ± 1.49	-422.51 ± 1.87
Keep-away	98.27 ± 0.15	90.48 ± 0.21	81.68 ± 0.29	80.22 ± 0.25

TABLE IV
GLOBAL REWARD IN THE LAST 10 000 EPISODES OVER TEN SEEDS. ± CORRESPONDS TO 95% CONFIDENCE INTERVAL OVER SEEDS. (AGENTS NUMBER = 10)

Environment	DD-MADDPG	DE-MADDPG	TD3	MADDPG	DDPG
Cooperative navigation	-4408.51 ± 3.50	-7532.22 ± 6.66	-9921.93 ± 16.75	-9220.31 ± 19.38	-10294.91 ± 21.74
Keep-away	-12.23 ± 0.15	-15.88 ± 0.27	-22.85 ± 0.38	-21.05 ± 0.67	-24.95 ± 0.63

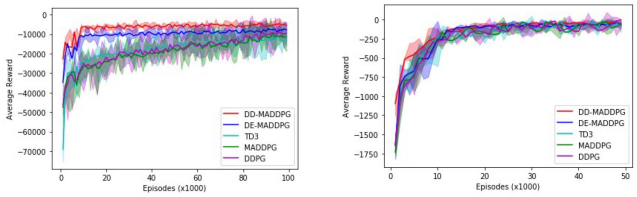
TABLE V
GLOBAL REWARD IN THE LAST 10 000 EPISODES OVER TEN SEEDS. ± CORRESPONDS TO 95% CONFIDENCE INTERVAL OVER SEEDS. (AGENTS NUMBER = 50)

Environment	DD-MADDPG	DE-MADDPG	TD3	MADDPG	DDPG
Cooperative navigation	-4960.81 ± 8.31	-8153.52 ± 11.90	-10565.93 ± 22.35	-11162.61 ± 23.71	-10424.61 ± 19.32
Keep-away	-54.18 ± 0.30	-63.43 ± 0.32	-82.65 ± 0.74	-64.65 ± 0.87	-73.85 ± 0.98



(a) (b)

Fig. 5. Graph of average global rewards with training episodes (agents number = 10). (a) Cooperative navigation. (b) Keep-away.



(a) (b)

Fig. 6. Graph of average global rewards with training episodes (agents number = 50). (a) Cooperative navigation. (b) Keep-away.

interval of each method in the last 10 000 episodes in the two environments as shown in Table III. The average reward of T2-PER is higher than other methods in the two environments, and the standard deviation is lower than others. Therefore, T2-PER can not only accelerate convergence but also improve training stability.

To evaluate the generalization ability of the proposed algorithm, we increased the number of agents in the cooperative navigation environment and keep-away environment. For Cooperative navigation, we expand the number of agents and lands from the original 3 to 10 and 50; for Keep-away, we expand the total number of agents from the original 3 to 15 and 75, and the number of predators from the original 2 is expanded to 10 and 50, and the number of land remains unchanged. Otherwise, the specific experimental method is the same as above. Fig. 5 shows the learning curve for evaluating DD-MADDPG.

It can be seen from Figs. 5 and 6 that the learning stability of the DD-MADDPG and DE-MADDPG algorithms is better than that the baseline algorithms, and the global reward obtained by the agent trained with the DD-MADDPG algorithm is significantly higher than other methods.To compare each method's specific performance, we quantify the performance of different methods into specific values, as shown in Tables IV and V. In the environment of ten agents, for the Cooperative navigation environment, the global reward of DD-MADDPG is about 50% higher than that of MADDPG, and the standard error is reduced by 84%. For the Keep-away environment, DD-MADDPG has the lowest standard error, which is reduced by about 76% compared to MADDPG. The evaluation results show that when the number of agents in the environment increases, the DD-MADDPG algorithm is still effective, and the stability and global reward agents achieved are better than other methods. In the environment of 50 agents, for the cooperative navigation environment, the collective return of DD-MADDPG is 52% higher than that of MADDPG, and the standard error is reduced by 58%. For the Keep-away environment, DD-MADDPG has the lowest standard error, which is reduced by about 65% compared to MADDPG. The evaluation results show that when the number of agents in the environment increases, the DD-MADDPG algorithm is still effective, and the stability and collective returns from agents are better than other methods.

The evaluation results of the T2-PER method are as follows: As can be seen from Figs. 7 and 8, the stability of T2-PER and TD-PER is generally higher than that of DD-MADDPG. To further measure the performance of different algorithms, we calculated the average return and standard error of each method after convergence in the two environments (according to Fig. 7, both Cooperative navigation and Keep-away converge after approximately 20 000 episodes), as shown in Tables VI and VII, in the two environments, the average

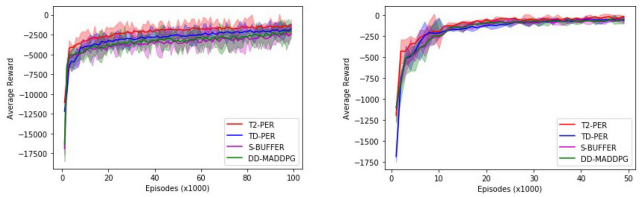
Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
GLOBAL REWARD IN THE LAST 10 000 EPISODES OVER TEN SEEDS. ± CORRESPONDS TO 95% CONFIDENCE INTERVAL OVER SEEDS. (AGENTS NUMBER = 10)

Environment	T2-PER	TD-PER	S-BUFFER	DD-MADDPG
Cooperative navigation	-1416.31 ± 2.07	1962.09 ± 2.18	-2583.21 ± 4.18	-3290.91 ± 3.76
Keep-away	-6.78 ± 0.15	-10.75 ± 0.19	-18.33 ± 0.09	-16.73 ± 0.18

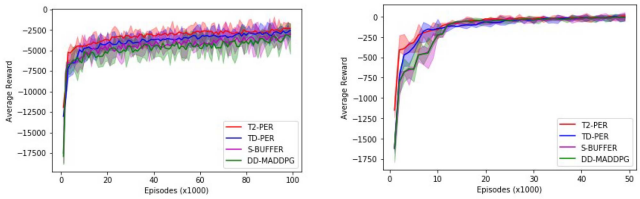
TABLE VII
GLOBAL REWARD IN THE LAST 10 000 EPISODES OVER TEN SEEDS. ± CORRESPONDS TO 95% CONFIDENCE INTERVAL OVER SEEDS. (AGENTS NUMBER = 50)

Environment	T2-PER	TD-PER	S-BUFFER	DD-MADDPG
Cooperative navigation	-2487.31 ± 2.17	-2789.79 ± 3.03	-3415.31 ± 6.05	-3685.01 ± 6.24
Keep-away	-33.28 ± 0.14	-57.15 ± 0.15	-60.78 ± 0.15	-62.58 ± 0.14



(a) (b)

Fig. 7. Graph of average global rewards with training episodes (agents number = 10). (a) Cooperative navigation. (b) Keep-away.



(a) (b)

Fig. 8. Graph of average global rewards with training episodes (agents number = 50). (a) Cooperative navigation. (b) Keep-away.

reward of T2-PER is higher than other methods, and the standard error is not higher than others. The results show that when the number of agents in the environment increases, T2-PER is still effective, and the learning stability and the global reward agents achieved are better than the other methods.

V. CONCLUSION

This article analyzed the influence of other agents' strategies on current agent learning in the multiagent system. To reduce this influence, we proposed the DD-MADDPG, which takes the global evaluation and local evaluation of the agent into consideration simultaneously and adaptively adjusts the two evaluations according to the training process. It reduces the interference of other agents' strategies. To reduce the influence of other agents' outdated experience, we proposed TD-Error and Time-based experience sampling (T2-PER) based on DD-MADDPG. It reduces the impact of outdated experience by reducing the probability of outdated experience being selected.

We evaluated the proposed method in the MPE simulation environment. The results showed that compared with MADDPG, DDPG, and other related methods, both DD-MADDPG and T2-PER can improve the stability of agent learning, and DD-MADDPG can increase the reward of the agent and improve the learning efficiency. When the number of agents in the environment increases, the proposed method is still effective, proving that the proposed methods also have a certain generalization ability.

REFERENCES

[1] V. Mnih *et al*. "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, Feb. 2015.
[2] Y. Duan, J. Schulman X. Chen, R. Houthooft, and P. Abbeel, "Benchmarking deep reinforcement learning for continuous control," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1329–1338.
[3] G. Lample and D. S. Chaplo, "Playing FPS games with deep reinforcement learning," in *Proc. Assoc. Adv. Artif. Intell. (AAAI)*, 2017, pp. 2140–2146.
[4] T. Schaul, J. Quan, I. Antonoglou, and D. Silvar, "Prioritized experience replay," in *Proc. IEEE Int. Conf. Learn. Represent.*, 2016, pp. 1026–1040.
[5] H. V. Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. Assoc. Adv. Artif. Intell. (AAAI)*, 2016, pp. 2094–2100.
[6] T. D. Bruin, J. Kober, K. Tuyls, and R. Babuška, "The importance of experience replay database composition in deep reinforcement learning," in *Proc. Deep Reinforcement Learn. Workshop (NIPS)*, 2015, pp. 2201–2213.
[7] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Singapore, 2017, pp. 3389–3396.
[8] N. Kumar, S. S. Rahman, and N. Dhakad, "Fuzzy inference enabled deep reinforcement learning-based traffic light control for intelligent transportation system," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 4919–4928, Aug. 2021.
[9] R. Makar, S. Mahadevan, and M. Ghavamzadeh, "Hierarchical multi-agent reinforcement learning," in *Proc. 5th Int. Conf. Auton. Agents*, 2001, pp. 246–253.
[10] P. Janakaraj, P. Pinyoanuntapong, P. Wang, and M. Lee, "Towards in-band telemetry for self driving wireless networks," in *Proc. IEEE INFOCOM Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Toronto, ON, Canada, 2020, pp. 766–773.
[11] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems 30*. Red Hook, NY, USA: Curran, 2017, pp. 6379–6390.
[12] F. A. Oliehoek, M. T. J. Spaan, and N. Vlassis, "Optimal and approximate Q-value functions for decentralized POMDPs," *J. Artif. Intell. Res.*, vol. 32, no. 1, pp. 289–353, 2008.
[13] L. Kraemer and B. Banerjee, "Multi-agent reinforcement learning as a rehearsal for decentralized planning," *Neurocomputing*, vol. 190, pp. 82–94, May 2016.
[14] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 2974–2982.
[15] P. Sunehag *et al.*, "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proc. Int. Conf. Auton. Agents MultiAgent Syst.*, 2018, pp. 2085–2087.

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.

[16] T. Rashid, M. Samvelyan, C. S. D. Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4292–4301.
[17] K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi, "QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5887–5896.
[18] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. 4th Int. Conf. Learn. Represent. (ICLR)*, 2016, pp. 1331–1342.
[19] H. U. Sheikh and L. Bölöni, "Multi-agent reinforcement learning for problems with combined individual and team reward," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2020, pp. 1–8.
[20] J. Foerster, I. A. Assael, N. de Freitas, and S. Whiteson, "Learning to communicate with deep multi-agent reinforcement learning," in *Advances in Neural Information Processing Systems 29 (NIPS)*. Red Hook, NY, USA: Curran, 2016, pp. 2137–2145.
[21] J. Z. Leibo, V. Zambaldi, M. Lanctot, J. Marecki, and T. Graepel, "Multi-agent reinforcement learning in sequential social dilemmas," in *Proc. 16th Int. Conf. Auton. Agents Multi-Agent Syst. (AAMAS)*, 2017, pp. 464–473.
[22] L. Matignon, G. J. Laurent, and N. L. Fort-Piat, "Hysteretic Q-learning : An algorithm for decentralized reinforcement learning in cooperative multi-agent teams," in *Proc. Int. Conf. Intell. Robots Syst. (IROS)*, San Diego, CA, USA, 2007, pp. 64–69.
[23] L. Panait, K. Sullivan, and S. Luke, "Lenient learners in cooperative multiagent systems," in *Proc. 5th Int. Joint Conf. Auton. Agents multi-Agent Syst. (ACM)*, 2006, pp. 801–803.
[24] S. Omidshafiei, J. Pazis, C. Amato, J. P. How, and J. Vian, "Deep decentralized multi-task multi-agent reinforcement learning under partial observability," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2681–2690.
[25] G. Palmer, K. Tuyls, D. Bloembergen, and R. Savani, "Lenient multi-agent deep reinforcement learning," in *Proc. Int. Conf. Auton. Agents Multiagent Syst.*, 2018, pp. 443–451.
[26] Y. Wang and Z. Zhang, "Experience selection in multi-agent deep reinforcement learning," in *Proc. IEEE 31st Int. Conf. Tools Artif. Intell. (ICTAI)*, Portland, OR, USA, 2019, pp. 864–870.
[27] D. P. Kingma and J. L. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent.*, 2015, pp. 1–14.
[28] S. Fujimoto, H. V. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1582–1591.
[29] J. Ackermann, V. Gabler, T. Osa, and M. Sugiyama, "Reducing overestimation bias in multi-agent domains using double centralized critics," in *Proc. NeurIPS Deep RL Workshop*, 2019, pp. 2875–2889.

Wei Pan received the Ph.D. degree from Northwestern Polytechnical University, Xi'an, China, in 2008.

He is an Associate Professor with the School of Computer Science, Northwestern Polytechnical University. His research interests include intelligent robots, decision support systems, multiagent systems, and machine learning.

Nanding Wang received the B.S. degree from the School of Information and Computing Science, Northwest A&F University, Xianyang, China, China, in 2019. She is currently pursuing the master's degree with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China.

Her research interests include deep reinforcement learning, multiagent systems, and intelligent decision systems.

Chenxi Xu received the B.S. degree from the School of Software and Microelectronics, Northwestern Polytechnical University, Xi'an, China, in 2019, where he is currently pursuing the master's degree with the School of Computer Science.

His research interests include mobile robot, deep reinforcement learning, and intelligent control.

Kao-Shing Hwang (Senior Member, IEEE) received the M.M.E. and Ph.D. degrees in electrical and computer engineering from Northwestern University, Evanston, IL, USA, in 1989 and 1993, respectively.

He is a Distinguished Professor with the Electrical Engineering Department, National Sun Yat-sen University, Kaohsiung, Taiwan, and was awarded the A.S.E. Chair Professorship in 2020. He is also an Adjunct Professor with the Department of Healthcare Administration and Medical Informatics, Kaohsiung Medical University, Kaohsiung, and a Visiting Chair Professor with the Department of Computer Engineering, Northwestern Polytechnical University, Xi'an, China. He had been with National Chung Cheng University, Minxiong, Taiwan, from 1993 to 2011, where he was the Deputy Director of the Computer Center from 1998 to 1999, the Chairman of the Electrical Engineering Department from 2003 to 2006, and the Director of the Opti-Mechatronics Institute from 2010 to 2011. His research interests include methodologies and analysis for various intelligent systems, visual servoing, and reinforcement learning for medical-aid systems and robotic applications.

Prof. Hwang received the MOST 2020 Outstanding Research Award, Taiwan. He is a Fellow of the Institution of Engineering and Technology.

Authorized licensed use limited to: Somaiya University. Downloaded on September 08,2025 at 10:37:31 UTC from IEEE Xplore. Restrictions apply.